Nexus SQL Validator (SQLite Version) – Detailed Overview

Team Members-

1.Deekshitha M

2.B Uday Kumar

3.Saketh Reddy

4.Sathvik K

5.Minakshi M

6.Pradhyuman Singh

7.Saqlain Ahmed P

This document explains the **SQLite-based SQL Validator service** you just set up in `sql_validator_agent_sqlite`：

1. High-Level Purpose

The project provides a **safe SQL validation and execution microservice** for an academic database schema.

- **Input:** An SQL query string (typically generated by another agent or service).
- **Processing:**
  - Parses and validates the query for:
    - Syntax
    - Semantics (tables exist)
    - Data range (year/semester bounds)
    - Security (dangerous patterns)
  - Optionally executes the query against a local SQLite DB.
- **Output:**
  - Structured JSON with per-check results.
  - Optionally, actual data rows when the query is approved.

This is ideal for putting in front of a **Query Generator Agent (QGA)** so that generated SQL is vetted before being run.

2. Project Structure (SQLite Agent)

Directory: `sql_validator_agent_sqlite/`

- **app.py**
  FastAPI application with the HTTP API:
  - `POST /validate`
  - `POST /validate_and_run`
- **validator.py**
  SQLValidator class encapsulating all validation logic.
- **init_db.sql**
  SQLite-compatible DDL + sample data for tables:
  - `Student`
  - `Semester`
  - `Subjects`
  - `Marks`
  - `Timetable`
- **init_db.py**
  Python script that creates `academic.db` and loads init_db.sql into it (no external sqlite3 CLI required).
- **evaluate.py**
  Batch evaluator that fires many candidate queries at the validator via HTTP.
- **test_validator.py**
  Pytest unit tests against the SQLValidator class.
- **smoke_test.py**
  Simple script that hits `/validate` and checks a handful of queries.
- **master_demo.py**
  "Master" script that:
  - Calls `/validate_and_run`
  - Shows checks and data
  - Prints human-readable reasoning for approval/rejection.
- **requirements.txt**
  Python dependencies (FastAPI, SQLAlchemy, etc.).
- **README.md**
  Setup and usage instructions (SQLite-specific).

Database file:

- **`academic.db`**
  SQLite database created/initialized by init_db.py.

3. Core Components

3.1 FastAPI app (app.py)

- **DB URI configuration:**

```
DB_URI = "sqlite:///academic.db"
```

- **App & validator:**

```
app = FastAPI()
validator = SQLValidator(DB_URI)
```

- **Request model:**

```
class QueryRequest(BaseModel):
    query: str
```

These pieces wire HTTP requests to the SQLValidator instance.

3.2 SQLValidator (validator.py)

The SQLValidator class is the core of the logic:

- **Initialization:**

```
self.engine = create_engine(db_uri)
self.metadata = MetaData()
self.metadata.reflect(bind=self.engine)
self.inspector = inspect(self.engine)
```

  - **`engine`** : SQLAlchemy engine connected to `academic.db` .
  - **`metadata.reflect`** : Reads the DB schema so the validator knows what tables exist.

- **Checks implemented:**
  - validate_syntax(query)
    - Uses the DB's SQL parser via:

```
conn.execute(text(f"EXPLAIN {query}"))
```

    - If this throws an exception → Syntax error.

- validate_semantics(query)
  - Uses `sqlparse` to parse the SQL and extract identifiers (table names).
  - Compares extracted names to `self.metadata.tables`.
  - If none match → "No valid tables referenced".
- validate_data_range(query)
  - Regex-based checks on `year` and `semester` in `WHERE` clauses:
    - `year` must be in `{1, 2, 3, 4}`
    - `semester` must be in `{1, 2, 3, 4, 5, 6, 7, 8}`
- validate_security(query)
  - Looks for dangerous patterns / keywords:
    - Examples: `drop`, `delete`, `insert`, `update`, `union`, `exec`, `--`, `;`
  - If any are found → security check fails.
- validate(query)
  - Runs checks in order:

    a. Syntax
    b. Semantics
    c. Data Range
    d. Security
  - Collects results into a list of:

    ```
    {"check": <name>, "valid": <bool>, "message": <str>}
    ```
  - Returns:
    - `(True, results)` if all pass.
    - `(False, results)` stopping at the first failure.

4. Endpoints

4.1 `POST /validate`

Purpose:
Validate a query and return per-check results, but **do not execute** the query.

- **Request body:**

```
{
  "query": "SELECT name, email FROM Student WHERE year = 1 AND semester = 1"
}
```

- **Success response (200):**

```
{
  "valid": true,
  "message": "Query is valid",
  "results": [
    {
      "check": "Syntax",
      "valid": true,
      "message": "Syntax valid"
    },
    {
      "check": "Semantics",
      "valid": true,
      "message": "Semantics valid"
    },
    {
      "check": "Data Range",
      "valid": true,
      "message": "Data range valid"
    },
    {
      "check": "Security",
      "valid": true,
      "message": "Security valid"
    }
  ]
}
```

- **Failure response (400):**

```
{
  "detail": {
    "valid": false,
    "results": [
      {
        "check": "Data Range",
        "valid": false,
        "message": "Invalid year value (must be 1-4)"
      }
    ]
  }
}
```

Usage:
Good for upstream agents that only need to know **if a query is acceptable**, without actually fetching data.

4.2 `POST /validate_and_run`

Purpose:
Validate a query and, if it passes all checks, **execute it** against `academic.db` and return rows.

- **Request body:** same structure as `/validate`.
- **On success (all checks pass + execution OK):**

```
{
  "valid": true,
  "message": "Query is valid",
  "results": [
    { "check": "Syntax", "valid": true, "message": "Syntax valid" },
    { "check": "Semantics", "valid": true, "message": "Semantics valid" },
    { "check": "Data Range", "valid": true, "message": "Data range valid" },
    { "check": "Security", "valid": true, "message": "Security valid" }
  ],
  "rows": [
    { "name": "Alice", "email": "alice@dsc.edu" },
    { "name": "Peggy", "email": "peggy@dsc.edu" }
    // ...
  ]
}
```

- **On validation failure (e.g., bad year or SQL injection):**

```
{
  "detail": {
    "valid": false,
    "results": [
      {
        "check": "Data Range",
        "valid": false,
        "message": "Invalid year value (must be 1-4)"
      }
    ]
  }
}
```

- **On execution-time error despite validation passing (rare):**

```
{
  "detail": {
    "valid": false,
    "results": [
      { "check": "Syntax", "valid": true, "message": "Syntax valid" },
      { "check": "Semantics", "valid": true, "message": "Semantics valid" },
      {
```

```
            "check": "Execution",
            "valid": false,
            "message": "some sqlite3/SQLAlchemy error message"
          }
        ]
      }
    }
```

Usage:
Best when you want **both** safety (validation) and actual **data retrieval** in one call.

5. Supporting Scripts

5.1 Database initialization (init_db.py)

- **What it does:**
  - Reads init_db.sql.
  - Connects to `academic.db` (creates it if missing).
  - Executes the full SQL script via `sqlite3` Python module.
- **Why it exists:**
  Avoids relying on a system-level `sqlite3` CLI. Works out-of-the-box with Python only.
- **Typical usage:**

  ```
  python init_db.py
  ```

  You'll see:

  ```
  Database initialized at: ...\academic.db
  ```

5.2 Batch evaluator (evaluate.py)

- **Purpose:**
  Sends a fixed list of candidate queries to `POST /validate`, collects stats on how many are valid vs invalid.
- **Outputs:**
  - Per-query log: HTTP status, validity, checks.
  - Final JSON summary:

```
{
  "total": 10,
  "valid": 5,
  "invalid": 5
}
```

- **Use case:**
  Quickly benchmark how your current validation configuration behaves on a suite of test queries.

5.3 Smoke tests (smoke_test.py)

- **Purpose:**
  Quick sanity check for the `/validate` endpoint.
- **What it does:**
  - Tests:
    - A valid query
    - Invalid year
    - Nonexistent table
    - SQL injection
    - Syntax error
  - Compares expected vs actual `valid` field.
  - Prints PASS/FAIL for each and a summary.
- **Use case:**
  Verify that changes to code or schema didn't break the basic validation contract.

5.4 Master demo (master_demo.py)

- **Purpose:**
  A **human-readable, narrative-style** demonstration of the end-to-end system, calling `/validate_and_run`.
- **Scenarios included:**
  - `valid_basic` (approved, with data rows printed).
  - `invalid_year`
  - `nonexistent_table`
  - `sql_injection`
  - `syntax_error`
- **Key behavior:**

- Prints:
  - Scenario name & description.
  - Query.
  - HTTP status.
  - Per-check results.
  - **Decision** line with natural language reasoning:
    - e.g.
      `APPROVED: all validation checks passed. Query is safe to run.`
      `REJECTED: Data range check failed: year/semester values are outside the allowed ranges (year 1-4, semester 1-8) (...).`
- Prints real data rows only when the query is approved.

- **Use case:**
  Great for demos, debugging, and explaining the system to others.

6. Typical Workflow

6.1 Setup

- **Create and activate venv** (you used `myvenv`):

```
cd sql_validator_agent_sqlite
python -m venv myvenv
.\myvenv\Scripts\Activate
pip install -r requirements.txt
```

- **Initialize DB:**

```
python init_db.py
```

- **Run API:**

```
python app.py
```

6.2 Using Postman

- **To validate only:**
  - Method: `POST`
  - URL: `http://localhost:8000/validate`
  - Body:

```
{
  "query": "SELECT name, email FROM Student WHERE year = 1 AND semester = 1"
}
```

- **To validate and fetch data:**
  - Method: `POST`
  - URL: `http://localhost:8000/validate_and_run`
  - Same body as above.

Postman is ideal for manually probing queries and seeing full JSON responses.

7. Integration with a Query Generator Agent

At a high level, your Query Generator Agent (QGA) can:

1. Generate a candidate SQL query from a natural-language request.
2. **Call `/validate`** (or `/validate_and_run`) with that query.
3. Read the `results` array:
   - If any check fails (`valid: false`), use the `message` fields as **feedback** to refine the next query.
4. Repeat until:
   - A query is **approved (`valid: true`)**, then:
     - Either execute it against the DB directly, or
     - Use `/validate_and_run` to both validate and execute.

You already have patterns for this both in:

- evaluate.py (simple loop).
- master_demo.py (reasoning about the checks).